

机器鱼中央控制系统

从设备接入、视觉识别到安全运动控制的一体化技术方案

技术介绍 · 2026.09



浏览器 → Go → Python / ESP32

先把问题说清楚：控制一条鱼，实际是管理一套系统

项目的核心不是“让舵机动起来”，而是把设备、视觉、权限、网络 and 恢复机制放到同一条可验证链路里。

设备层

ESP32 负责运动执行、心跳、设备身份、联网、状态灯和 OTA。

控制层

Go 是唯一控制入口，统一接入设备、视觉、用户会话和命令队列。

感知层

Python 负责摄像头、YOLO、跟踪、坐标映射、路径和视觉控制计算。

项目要解决的四个技术问题

1 统一入口

浏览器不直连 ESP32，所有命令经过 Go。

2 闭环控制

视频、识别、控制量和设备回报形成闭环。

3 安全边界

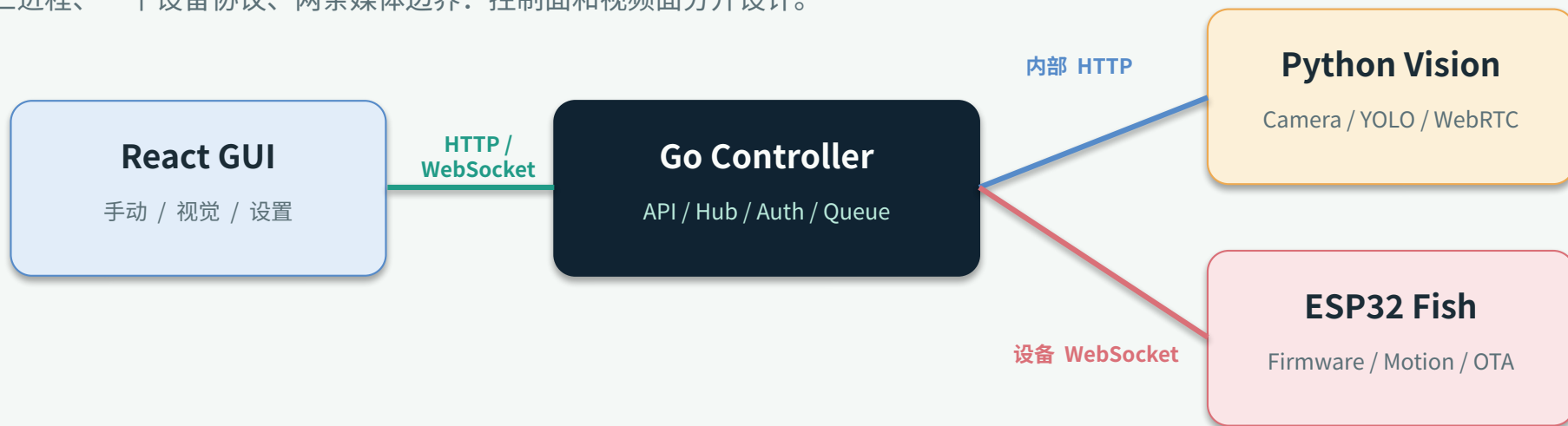
控制权、超时、急停、断线保护分别成立。

4 可演进

OTA 与 Mixly 风格编程复用现有能力和安全链路。

总体架构：浏览器只面对 Go ， Go 负责秩序

三进程、一个设备协议、两条媒体边界：控制面和视频面分开设计。



关键边界

- Python 不直接访问鱼的 IP ；视觉运动意图必须经过 Go 。
- WebRTC 传视频，Go 代理信令与 API ，不把视频字节塞进设备 WebSocket 。
- ESP32 只执行最终运动参数，不接收视频、PID 或网页身份。

一次“前进”命令，如何走完端到端链路

每台设备拥有独立发送队列；状态以设备回报为准。



协议层的三个语义

queued	已进入控制器队列，不代表设备已执行
sent	已写入设备 WebSocket
acknowledged	设备返回同 requestId 的结果

视觉不是“识别完就发命令”，而是一条有身份绑定的闭环

YOLO 负责低频身份确认，标记 / 跟踪负责更高频观测，最终控制量仍回到 Go 的设备队列。



视觉身份 ≠ 物理设备身份

必须显式绑定

识别目标 ID 与 ESP32 deviceId 不能靠“看起来像”自动对应。

工作区隔离

每个账号 + 浏览器 + 鱼拥有独立视觉控制工作区。

失效即停止

目标丢失、会话过期、摄像头失帧或控制权变化都触发停止。

安全设计：把“停止”做成多层防线

不同超时解决不同问题，不能用一个 60 秒租约替代设备心跳或视觉运动保护。

设备心跳	约 3 秒	设备 / Go 断连后停止
视觉运动有效期	约 3 秒	Python 卡住或视觉失效时停止
手动控制租约	默认 60 秒	控制权归属与接管，不等于持续运动
舵机参数限幅	中位 ± 振幅	Go 联合校验，避免超出标定范围

认证链路 网页账号 → 浏览器 clientId → 设备租约 → HMAC challenge / response → WebSocket 注册

多用户控制：账号拥有设备，不代表当前窗口可以发运动

控制归属和控制会话拆开，避免刷新、换窗口或接管后旧命令继续生效。

状态转换



旧窗口命令被拒绝
停止命令仍优先入队

控制权判断的三个条件

- ✓ `userId` 匹配当前登录账号
- ✓ `clientId` 匹配当前浏览器窗口
- ✓ `deviceId` 对应当前设备租约

工程意义

刷新页面可以恢复登录，但不能自动恢复运动会话；恢复控制必须显式发生。

OTA 与图形化编程：两类任务，共用一条受控执行链

固件升级改变设备能力；临时程序编排设备行为。二者都不能绕过 Go 的安全闸门。

OTA：改变固件能力

管理员上传 firmware.bin + manifest

预检查：硬件 / 能力 / 版本 / SHA-256

升级前停止运动 → 写入备用分区 → 重启 → 重新注册确认

失败时保留身份配置，并进入回滚或待核查流程。

Mixly 风格：编排临时行为

学生程序只调用底层固件的白名单能力

运动、灯光、传感器、条件、循环和计时

程序运行在控制器内存，不写入固件分区

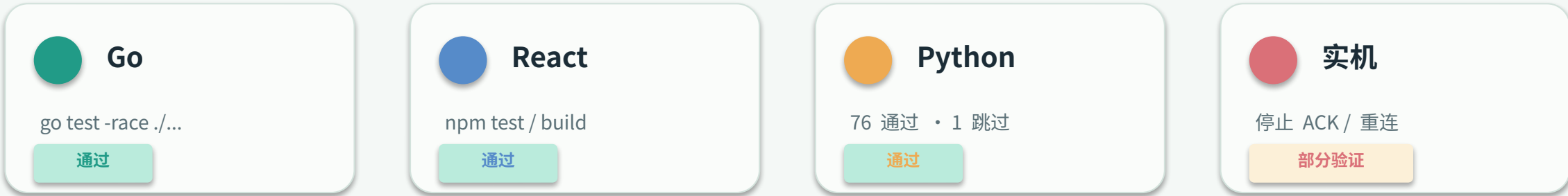
停止、急停、断线或重启后，运行实例自动清除。

同一控制器

共享：能力描述、设备队列、控制权、运动限幅、超时保护、急停

工程验证：自动化已经覆盖核心逻辑，实机边界仍需诚实标注

截至 2026 年 9 月 4 日维护记录，项目已形成 Go、前端、Python、固件测试的组合验证。



已验证的能力

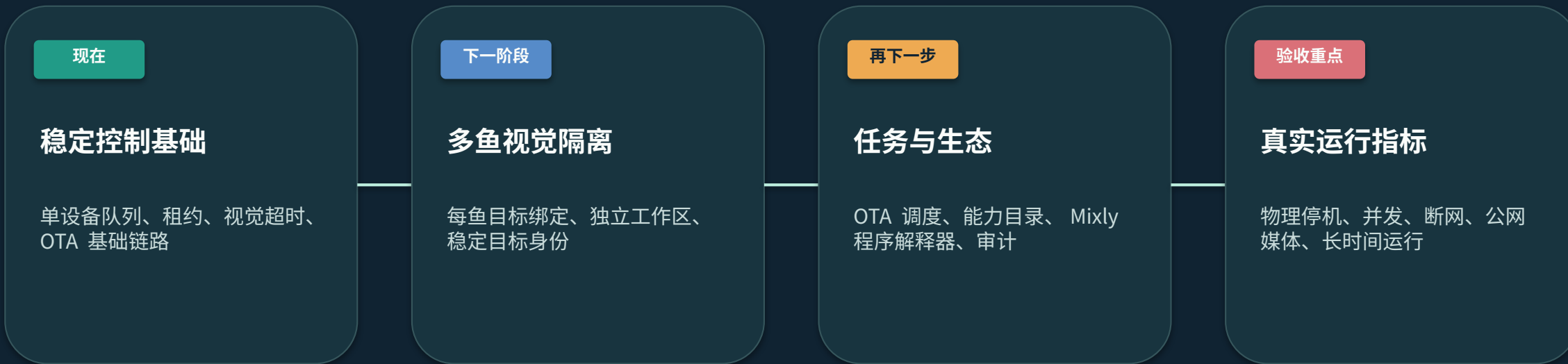
- 视觉超时停止、旧会话拒绝、租约接管和发送队列生命周期。
- 相机裁剪、WebRTC/API、设备发现、认证、运动限幅和状态恢复。
- 本机部署健康检查：Go 8081、Python 8091、前端 8098。

尚待实机验收

- 带载舵机边界与固件中位和 Go 标定的一致性。
- 运行中断网 / 停 Python 的物理停机时延。
- 两台真实电脑接管、多鱼并行控制和公网媒体长时间稳定性。

下一步：从“能控制”走向“可规模化运行”

建议按依赖顺序推进，先收敛身份和任务模型，再扩展并发与生态。



演示建议

设备发现 → 领取控制权 → 手动动作 → 启动视觉 → 目标绑定 → 视觉停止保护 → OTA 预检查

一句话总结：Go 把设备、视觉和用户控制收敛为一条可验证、可恢复、可扩展的控制链路。